

# CS 35L Software Construction

## Lecture 12 – Design With Reuse

**Tobias Dürschmid**  
Assistant Teaching Professor  
Computer Science Department



# This Lecture – Design with Reuse

---

- What are **advantages** of reusing existing modules?
- What **challenges** might arise from reusing existing modules?
- How to **decide** whether to reuse a module?
- How to **reduce the risk of** negative consequences of reuse?
- How to **make & document design decisions**?

# Why Reuse? (Instead of Re-Implementing)



## Higher Productivity / Faster Time to Market

Reusing software can **speed up software development** because time for implementation and testing may be reduced.

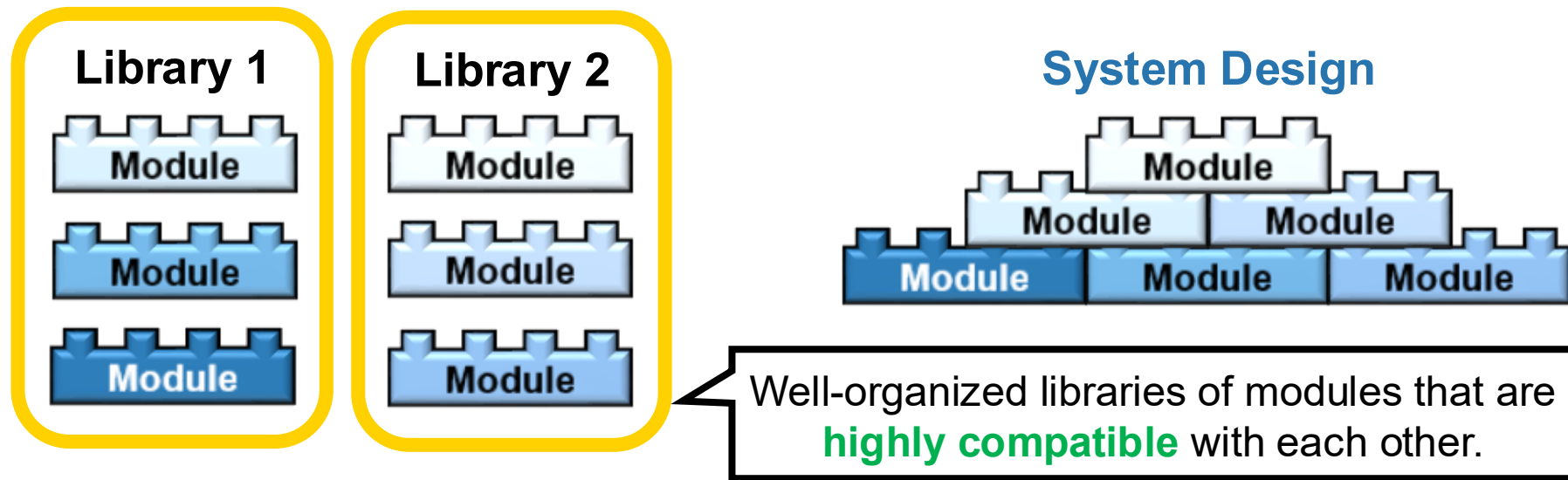


## Higher Software Quality / Fewer Defects

Reused software, which has been **tried and tested** in working systems, should be **more dependable** than new software, since most bugs have likely been found already by other users of the module.

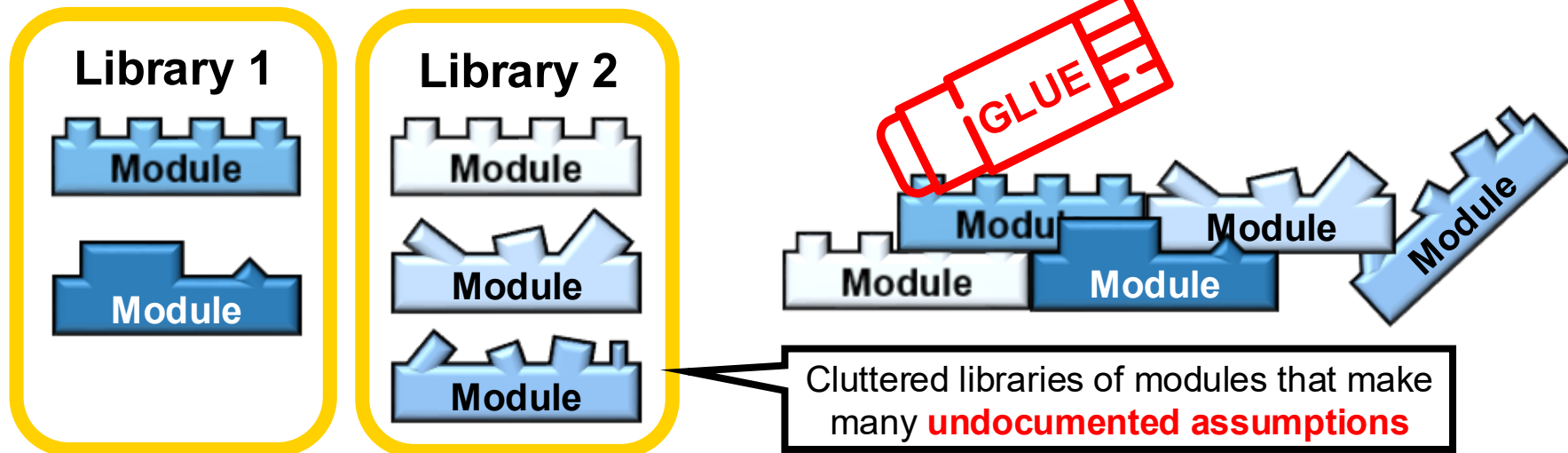
See [“What software reuse benefits have been transferred to the industry? A systematic mapping study”](#) (José L. Barros-Justo et al. 2017)

# The Vision of Reuse: Creating New Software Mostly by Composing Existing Building Blocks



Read more in: "[Mass Produced Software Components](#)" (Malcolm Douglas McIlroy 1968)

# The Reality of Reuse: Modules are Partially Incompatible But Often Still Glued Together



Read more in: "Architectural Mismatch: Why Reuse Is (Still) So Hard" (David Garlan et al. [1995](#) and [2009](#))

# Reuse must be Approached Differently Depending on its Source



## Internal Reuse

Code was written by the **same developer, team, or organization** that is reusing it (e.g., product lines, component-based development process, ...)



## External Reuse

Code was written by a **third party**.  
(e.g., commercial off-the-shelf, open-source libraries, packages, frameworks)



# How to Design with External Reuse?

Design With Reuse

CS35L Software Construction

# The Python Ecosystem Is Built on Reuse

**Most commonly needed functionality** is already implemented in a reusable way

**Low Entry Barrier:** Importing & starting to use reusable modules is easy: `$ pip install requests`

```
>>> import requests
>>> response = requests.get("https://api.github.com")
>>> response.status_code
200
>>> response.json()
{'current_user_url': 'https://api.github.com/user',
...}
```

## Python Ecosystem



# Example: Python Package Update Has **API-Breaking Change**

How can we prevent this from happening?

**Context:** No source code changes

Python's `docker` package imports the `requests` package and the `urllib3` package

**Error Message:** `docker.errors.DockerException:`  
`Error while fetching server API version: request()`  
`got an unexpected keyword argument 'chunked'`

**Root Cause:** `urllib3 2.0.0` just released today! And it changed its API to be incompatible with `requests`

```
// in requests package
```

```
httplib_response  
= self._make_request(  
    conn,  
    method,  
    url,  
    timeout=timeout_obj,  
    body=body,  
    headers=headers,  
    chunked=chunked,  
)
```



# Design Principle: Keep Versions of Your Dependencies Fixed

- Most package managers allow you to **specify the versions of dependent packages** & install them in a virtual environment locally to the project
- E.g., Python: Use Pipenv & Pipfiles

~~\$ pip install requests~~

~~\$ python <program>~~



See more here: <https://pipenv.pypa.io/>

## Example Pipfile

```
[packages]
urllib3 = "<2.0.0"
docker = "==7.1.0"

[dev-packages]
pep8-naming = "==0.10.0"
mypy = "==0.910"
pytest = "==5.4.2"
tox = "==3.15.1"

[requires]
python_version = "3.9"
```

\$ pipenv install requests

\$ pipenv run <program>



# Heartbleed Bug In OpenSSL

Reusable Packages can  
introduce Security Vulnerabilities



What can we learn from this bug? Talk to your Neighbor(s)

Encrypted Connection via SSL/TLS

Component A

OpenSSL

Component B

Introduced in February 2012

Discovered on 1 April 2014

Fixed version released on 7 April 2014

Included **insecure**  
**implementation** of *Heartbeat*  
leading to a buffer over-read,  
leaking memory data

7 April 2014      17% of all secure web servers vulnerable

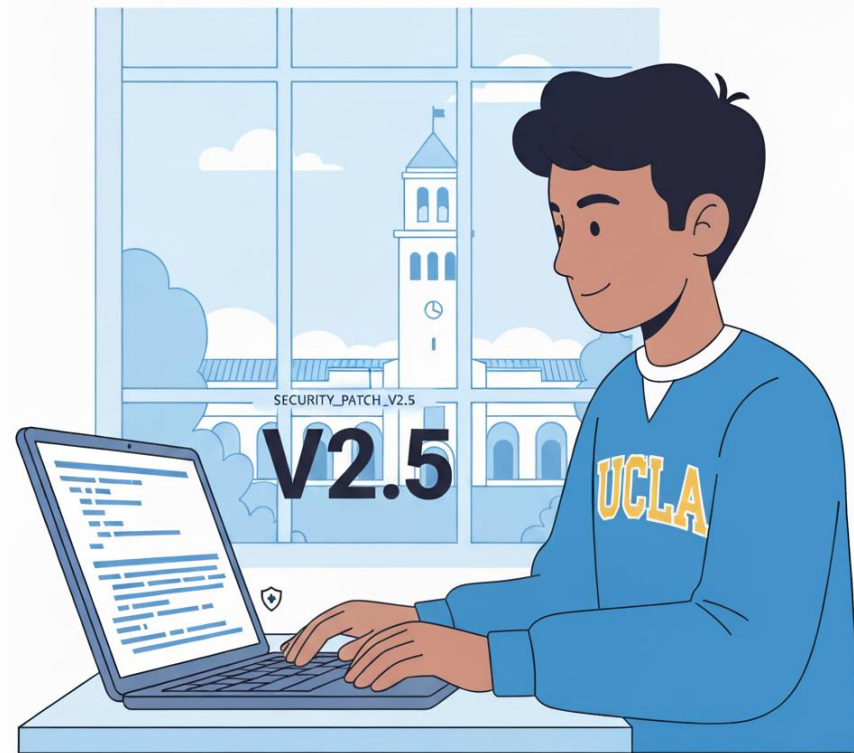
20 May 2014      1.5% of most popular TLS-enabled websites still vulnerable

January 2017      180k internet-connected devices still vulnerable

July 2019      91k devices still vulnerable

# Design Principle: Update Your Dependencies To Receive Bug Fixes & Security Patches

- Reusing **well-maintained** modules can improve your software quality
- Defects in popular modules are usually **fixed quickly**
- **Regularly check** for security patches & bugfixes.
- Be aware of the **side effects** of updates (see previous example)



# left-pad – A Simple and Highly Reused npm Package

**left-pad** adds characters in front of a string for alignment with just 11 lines of code.

Transitively, used in big popular packages (e.g., **React**, **Babel**), which are used by most web apps.

```
module.exports = leftpad;  
function leftpad (str, len, ch) {  
  str = String(str);  
  var i = -1;  
  ch || ch = ' ';  
  len = len - str.length;  
  while (++i < len) {  
    str = ch + str;  
  }  
  return str;  
}
```

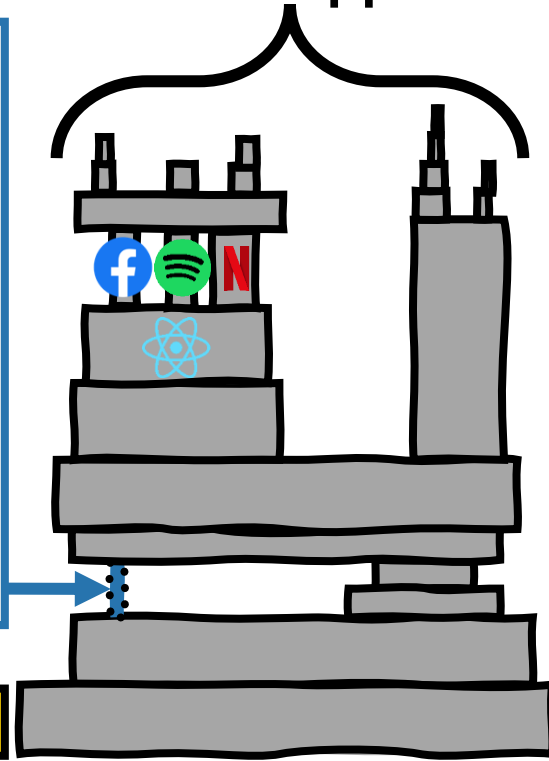
Stars on GitHub: 10

Weekly downloads:  $\approx$  1M

**left-pad**

How could we refactor this code?

Most Modern Web Apps



# left-pad – How Reusing Just 11 Lines Broke the Entire Internet

March 22, 2016: Due to a dispute with npm, the author of `left-pad` decides to **unpublish** all his packages (including `left-pad`)

Build processes for web apps across the internet **broke** due to the **missing package**

Many developers did not even know that they were **transitively relying** on `left-pad`

2 hours after the original `left-pad` package was removed, npm manually "un-unpublished" it.

Read more here: <https://www.davidhaney.io/npm-left-pad-have-we-forgotten-how-to-program/> & <https://qz.com/646467/how-one-programmer-broke-the-internet-by-deleting-a-tiny-piece-of-code>

Most Modern  
Web Apps



# Learning from the left-pad story, Describe Rules for Reusing Developers that Prevent Issues like that

Small-Group  
Discussion

Talk to your  
neighbor!

How should we decide  
**what to reuse?**

How can we **minimize**  
**the risk of reuse?**

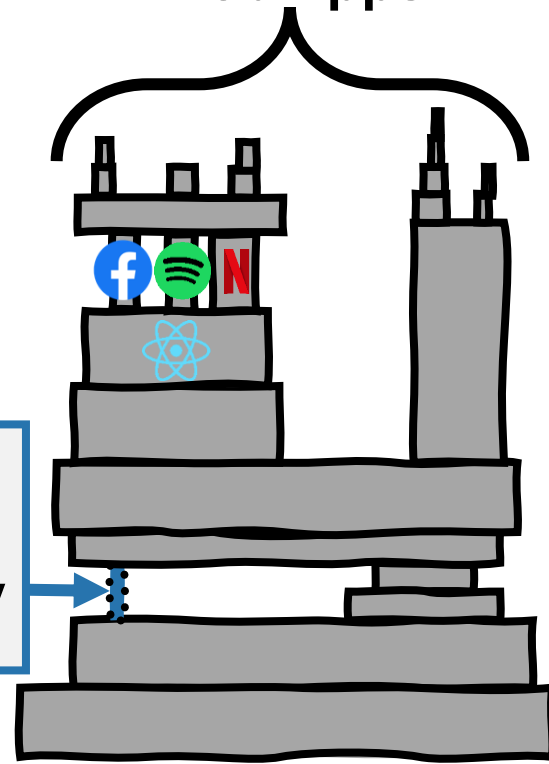
```
return toString.call(arr)  
== '[object Array]';
```

Stars on GitHub: 129

Weekly downloads:  $\approx$  92 million

**isArray**

Most Modern  
Web Apps



# Design Principle for Design With Reuse: Strive for Fewer Package Dependencies

---

- **Avoid reusing trivial code**, especially from unreliable sources
- **Carefully consider** adding new package dependencies
  - Every dependency can break, or **stop being supported**
  - Package dependencies can become a **security vulnerability** (e.g., eslint-scope malicious update)
- **Analyze your supply chain** for weaknesses and risks

See <https://eslint.org/blog/2018/07/postmortem-for-malicious-package-publishes/>

# Modules with higher Maintenance Level & Popularity Are more Viable Reuse Candidates

2 hours after the original `left-pad` package was removed, npm manually "**un-unpublished**" it, **because it broke the entire internet**

- How actively does the development team **fix bugs** and update the module to support **new platforms**? Do they commit often?
- **Popular** packages with many users are more likely to **resolve issues quickly** & have better **documentation**
- However, **fit to your context** is more important than popularity!

# Lesson Learned: External Reuse Is **Not a One-Time Investment!**

**Key-Takeaway from  
Previous Examples**

- **Important updates** (e.g., fix security vulnerabilities) might come with **API-breaking changes** if you have skipped previous versions.
- Poorly maintained packages might require you to **abandon them later**
- Relying too much on reused code **limits changeability** once you need more than what the library offers.

# Cost-Benefit Analysis for External Reuse

## Effort to adapt the reusable module

Integration Effort  
(**Complexity**, Similarity of **Context**)

Finding the Module

Updating Effort

Limiting **Changeability**



## Effort saved by reusing the module

Implementation Effort

Testing Effort

Benefit of **Update**  
Propagation

Read more here: [Why reinventing the wheels? An empirical study on library reuse and re-implementation \(Xu et al. 2019\)](#)

# Should You Reuse these Packages?

## Talk to Your Neighbor(s)

**Context:** Building an **appointment scheduling system**

Which of these packages are good reuse candidates? What are the pros and cons of reusing them? Decide for each package individually.

### python-constraint

Provides a simple constraint satisfaction problem (CSP) solver in Python to identify a scheduling solution for multiple users



Reusing a **hard-to-implement** functionality of medium size (1.3k LOC)



**Limits changeability** (what if we want priority scheduling instead of global optimization?)

### icalendar

Generates, parses, and manipulates iCalendar data to send invitations to users



Reusing a **large amount of hard-to-implement** functionality



**High changeability** due to local change impact



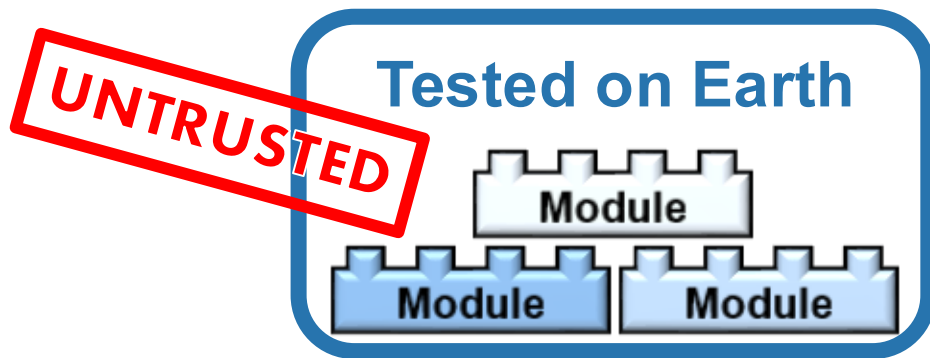
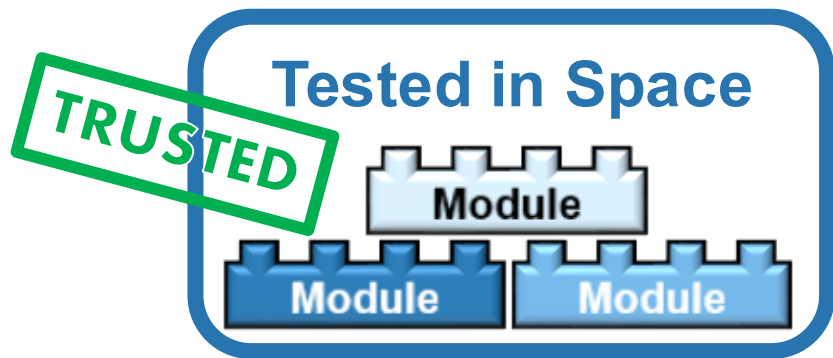
# How to Design with Internal Reuse ?

Design With Reuse

CS35L Software Construction

# NASA Heavily Relies on Internal Reuse

- **Problem:** Creating appropriate integration & system-level **tests** for space craft software is **difficult on Earth**
- **NASA's Solution:** Only trust software that has **worked in space**



# Ariane 5 Failure

**Describe Reuse Rules that Avoid Failures Like This**

**Ariane 4 Flight Control System**

**Worked  
Perfectly!**

Inertial Reference System

Horizontal Velocity - 16 Bit Int

Can reach **higher velocities** than Ariane 4

Assumed **lower velocity**

Due to **Performance Requirement**

**\$370M  
in 37s**

**Ariane 5 Flight Control System**

**Caused Self-  
Destruction**

Inertial Reference System

Horizontal Velocity - 16 Bit Int

Overflow Error

**Small-Group  
Discussion**

See <http://esamultimedia.esa.int/docs/esa-x-1819eng.pdf>



# Lesson Learned from Ariane 5: Software that Worked in one Context Might Not Work in Another Context



R5 Review all flight software (including embedded software), and in particular :

- **Identify all implicit assumptions** made by the code and its justification documents on the values of quantities provided by the equipment. **Check these assumptions** against the restrictions on use of the equipment.

See [https://www.esa.int/Newsroom/Press\\_Releases/Ariane\\_501\\_-\\_Presentation\\_of\\_Inquiry\\_Board\\_report](https://www.esa.int/Newsroom/Press_Releases/Ariane_501_-_Presentation_of_Inquiry_Board_report)

# Design Principle for Internal Reuse: Identify Violated Assumptions

---

- Check documentation and code to **identify assumptions** made by a reuse candidate
- Check to make sure that reusable software was designed to operate reliably **under the conditions you want**
- Don't **assume** the code of the reuse candidate is **correct; test it!**

# Cost-Benefit Analysis for Internal Reuse

---

**Effort to adapt** the  
reusable module

Identification of  
Implicit  
**Assumptions**

Effort to Create /  
Identify Reusable  
Modules



**Effort saved** by  
reusing the module

**Implementation** Effort

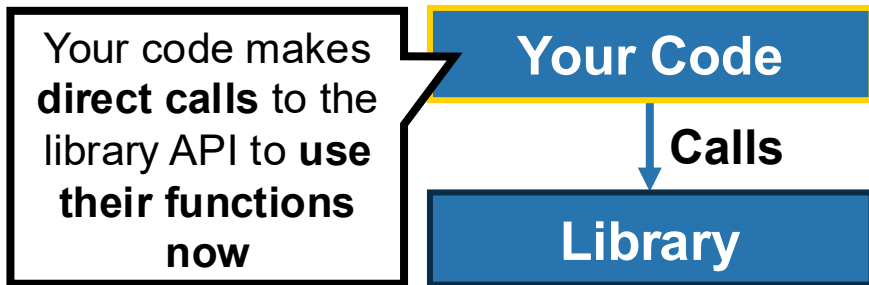
**Testing** Effort

Benefit of **Update**  
Propagation

# Library vs. Framework

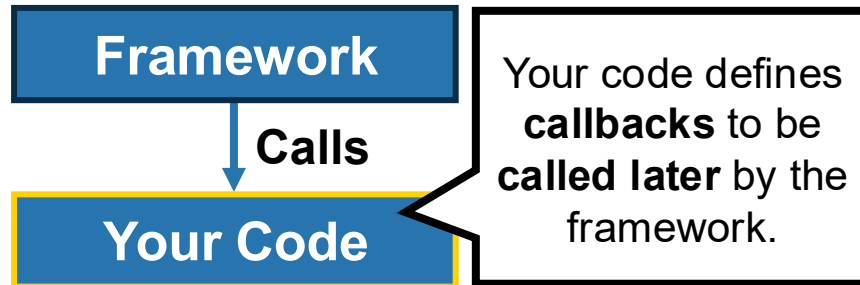
## “Hollywood Principle” of Inversion of Control

“Don’t call us. We’ll call you”. Frameworks primarily allow you to define callbacks



For example, in Axios:

```
const response = await axios.get  
    ('/user?ID=12345');  
console.log(response);
```



For example, in Express:

```
app.get('/', (req, res) => {  
    res.send('Hello World!')  
})
```

# Library vs. Framework

## “Hollywood Principle” of Inversion of Control

“Don’t call us. We’ll call you”. Frameworks primarily allow you to define callbacks



A **framework** makes more **decisions** for you and gives you **less flexibility**. But it also **hides a lot of complexity**, which often means **you write less code**. Hence, decisions to use a framework are often **harder to reverse** later.



# How to Make Design Decisions?

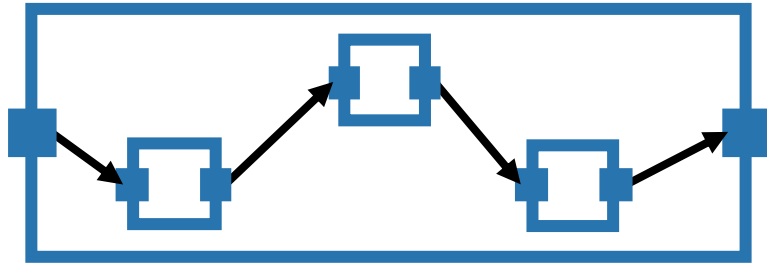
Design With Reuse

CS35L Software Construction

# Which Team Created A **Better** Design?

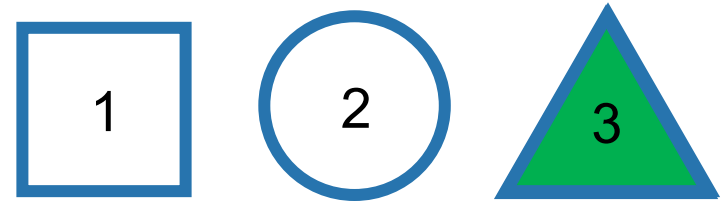
## Team A

Produced **one** detailed design option



## Team B

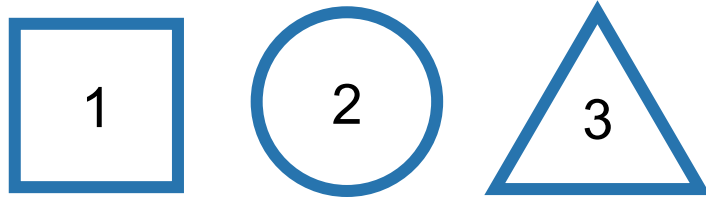
Produced **three** design options



# Which Team Created A **Better** Design?

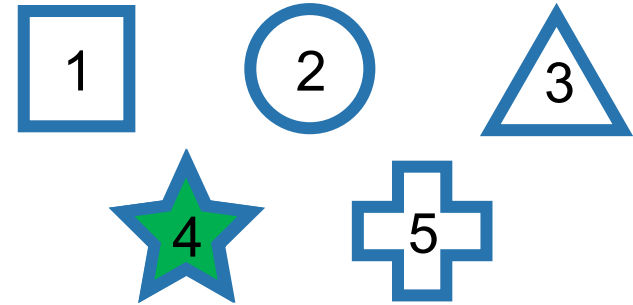
## Team B

Produced **three** design options



## Team C

Produced **five** design options



See [Petre, Marian "Insights from Expert Software Design Practice" in: European Software Engineering Conference and Symposium on The Foundations of Software Engineering \(ESEC/FSE 2-09\)](#)

# Lesson Learned:

## Think of **Many Design Alternatives**

---



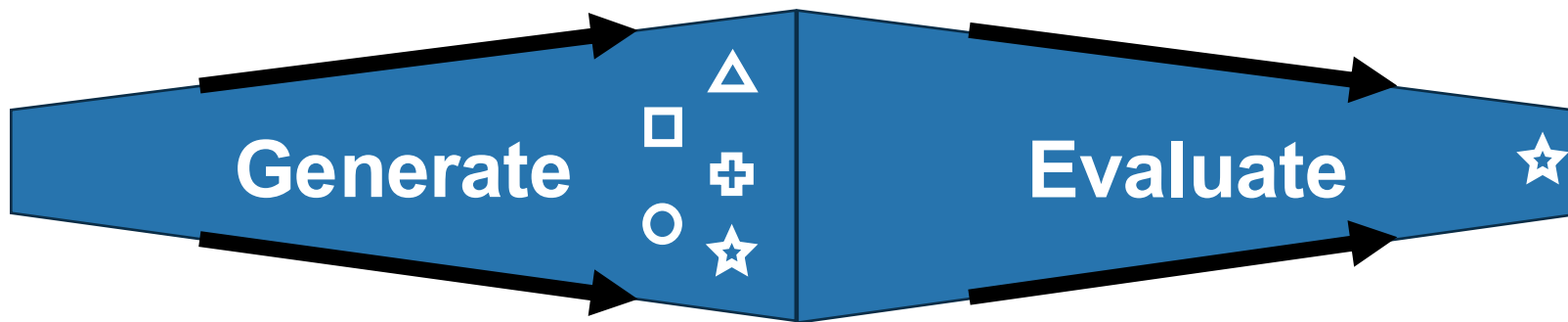
- If you can think of a good design, try to **think of a better one**
- Think broadly about a **diverse** range of solution
- Research has shown: When simply prompting designers to **consider other design alternatives**, designers with less experience create **better designs** [1]

[1] [Tofan, Dan, Matthias Galster, and Paris Avgeriou. "Difficulty of architectural decisions—a survey with professional architects." European Conference on Software Architecture \(ECSA 2013\)](#)

# Lesson Learned:

## Think of **Many Design Alternatives**

---



The purpose of idea  
**generation** is to **broaden up**

**Narrowing down** of ideas is  
done in the **evaluation** activity

# Delaying Decisions

---

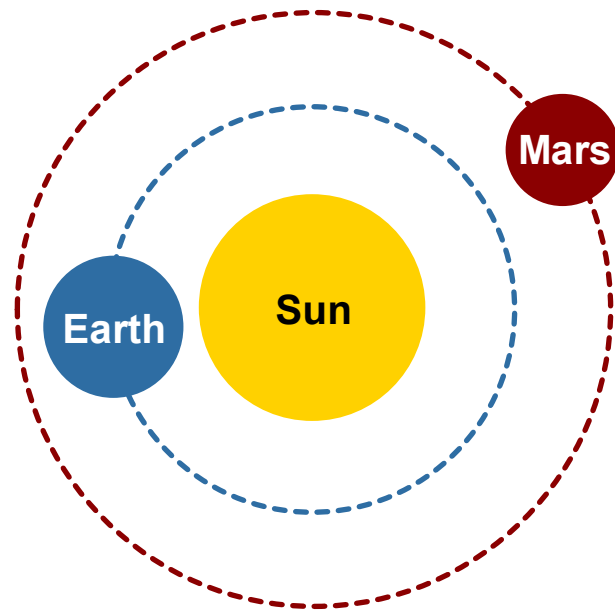
- Identify design decisions **that need more information** or that are likely to change later
- Attempt to **design your system without assuming a solution** for these difficult decisions
- Keep a **list of delayed decisions** and keep track of what you need to resolve them

# Design Exercise: Divide And Conquer

Design an interplanetary messaging system for people living on **Earth** and **Mars** to communicate!

What sub-problems would you need to solve?

Problems that arise when messaging on **Earth**, added infrastructure, networking with largely varied distances, and different definition of a day



# Lesson Learned: Solve Simpler Problems First

---

- When faced with a complex problem, experts solve a simpler problem first
- Solution to simpler problem might be incomplete but can be **extended later**
- Be aware when the simpler problem is so fundamentally different that solutions do not generalize

# Making Design Decisions (The “Rational” Decision Making Process)

Has been found to **lead to better design**  
(especially effective for early-career software engineers) [2]

**Step 1: Identify your requirements. That is important?**

**Step 2: Think of Many Design Alternatives**

**Step 3: Evaluate how well your Design Alternatives help you implement the Requirements from Step 1**

**Step 4: Consider the Trade-offs and Make a Decision**

[2] Tang, Antony, et al. "Design reasoning improves software design quality." *International Conference on the Quality of Software Architectures*. Springer. 2008.

# At Google, before the Implementation of a New System, the Developers writes a "Design Doc"

## Goal

- **Early identification of design issues** when making changes is still cheap.
- Achieving **consensus** around a design in the organization.
- **Transferring knowledge** from senior engineers into the organization.
- Form the basis of an organizational **memory around design decisions**.
- Acts as a summary artifact in the **technical portfolio** of the software designer(s).

“As **software engineers our job is** not to produce code per se, but rather **to solve problems**. Unstructured text, like in the form of a **design doc**, may be **the better tool for solving problems early in a project lifecycle**, as it may be more concise and easier to comprehend, and **communicates the problems and solutions at a higher level than code.**” – Malte Ubl

See <https://www.industrialempathy.com/posts/design-docs-at-google/>

# "Design Docs" Consist of These Parts:

**Context & Scope** bring the reader up to speed

**Background facts** that are relevant to understand the Design Doc

**Goals & Non-Goals** define the Scope

**Goals:** Requirements & quality attributes

**Non-Goals:** What can be ignored to simplify development?

**The Design** describes the selected solution

**Models & design descriptions** (e.g., context diagram, data model, API descriptions, pseudo-code, constraints, ...)

**Alternatives** motivate why this decision was made

Trade-offs that each respective design makes and how those trade-offs led to the decision to select the design that is the primary topic of the document.

See <https://www.industrialempathy.com/posts/design-docs-at-google/>

# Please fill out your Exit Tickets on Bruin Learn!

## Question 1

1 pts

In your own words, please summarize **three design principles** you learned about **Design With Reuse** today. (Do not copy phrases from the lecture slides)

## Question 2

1 pts

Describe the **pros** and **cons** of using the [axios](#) library versus the [fetch](#) API for HTTP requests in Node.JS. **Decide which one is more appropriate for your course project and why.**

## Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slide use images from Flaticon.com (Creators: Freepik, surang, Eklip Studio)